

Compte rendu hebdomadaire - Projet Tempo

Semaine du 31 août au 4 septembre 2026 Valentin Musset - Concepteur développeur d'applications

Cette semaine, j'ai travaillé sur la fiabilité des réservations, les invitations, le check-in par QR code et les tests automatisés de Tempo.

1. Réalisations

1.1. Réservations sans chevauchement

1.1.1. Affichage

L'utilisateur choisit un espace, un créneau et une visibilité publique ou privée. Si le créneau empiète sur une réservation existante, la page affiche : « Ce créneau est déjà réservé pour cet espace ».

1.1.2. Extrait de code

```
const overlapping = await db.query.bookings.findFirst({
  where: and(
    eq(bookings.workspaceId, workspaceId),
    lt(bookings.startAt, endAt),
    gt(bookings.endAt, startAt),
  ),
});
```

1.1.3. Argumentation

Le service détecte les intersections entre deux créneaux. Une contrainte PostgreSQL complète ce contrôle lorsque deux requêtes arrivent en même temps. L'API renvoie alors HTTP 409. Deux créneaux consécutifs restent autorisés.

1.2. Invitations et check-in par QR code

1.2.1. Affichage

Le propriétaire invite un utilisateur par email. L'invité accepte ou refuse depuis la page des réservations. Une fois accepté, il peut scanner le QR pendant le créneau et obtenir le message « Présence confirmée ».

1.2.2. Extrait de code

```
if (!participant) throw new Error('PARTICIPANT_NOT_FOUND');
if (participant.invitationStatus !== 'ACCEPTED') {
  throw new Error('INVITATION_NOT_ACCEPTED');
}
if (now < participant.booking.startAt) {
  throw new Error('CHECK_IN_TOO_EARLY');
}
```

1.2.3. Argumentation

Le backend vérifie l'identité, l'invitation et l'heure. Le QR contient un jeton aléatoire de 256 bits, mais PostgreSQL ne conserve que son hash SHA-256. La génération d'un nouveau QR invalide le précédent.

1.3. Capacité et concurrence dans PostgreSQL

1.3.1. Affichage

Une invitation en attente réserve une place. Si la capacité est atteinte, l'ajout d'un participant est refusé avec HTTP 409.

1.3.2. Extrait de code

```
await transaction.execute(
  sql`SELECT "id" FROM "bookings"
      WHERE "id" = ${bookingId} FOR UPDATE`,
);
const reservedPlaces = await countReservedPlaces(transaction, bookingId);
if (reservedPlaces >= booking.workspace.capacity) {
  throw new Error('BOOKING_FULL');
}
```

1.3.3. Argumentation

Le verrou FOR UPDATE empêche deux ajouts simultanés de dépasser la capacité. Le comptage et l'écriture restent dans la même transaction. Une contrainte unique empêche aussi d'ajouter deux fois le même utilisateur à une réservation.

1.4. Audit des suppressions avec MongoDB

1.4.1. Affichage

La page Journal d'audit affiche les 100 suppressions les plus récentes avec l'entité, la date et l'auteur.

1.4.2. Extrait de code

```
await this.log({
  action: actionMap[entityType],
  entityType,
  entityId,
  entityData,
  performedBy,
});
```

1.4.3. Argumentation

PostgreSQL garde les données métier. MongoDB reçoit les événements d'audit, dont la structure varie selon l'entité. L'écriture est en mode best effort : une panne MongoDB ne bloque pas une suppression PostgreSQL, mais un événement peut manquer.

2. Éléments de sécurité de l'application

Risque	Mesure appliquée
Mot de passe exposé	Hash avec Bun . password. Le hash n'est jamais renvoyé par l'API.
Accès non autorisé	JWT de 24 heures, authGuard et adminGuard. Un jeton absent donne HTTP 401 et un rôle insuffisant HTTP 403.
Donnée incorrecte	Validation Zod avant l'appel des services.
Double réservation	Contrainte d'exclusion PostgreSQL.
Dépassement de capacité	Transaction et verrou FOR UPDATE.
Brute force	10 requêtes par adresse sur 15 minutes, puis HTTP 429.
Jeton QR divulgué	Hash SHA-256, date d'expiration et rotation du jeton.
Mauvaise configuration	JWT_SECRET et FRONTEND_ORIGIN obligatoires au démarrage.

Le CORS n'accepte que FRONTEND_ORIGIN. L'API ajoute aussi une CSP et plusieurs en-têtes de sécurité. Les secrets restent dans les fichiers d'environnement.

Le JWT est encore stocké dans localStorage. Pour une mise en ligne publique, il faudrait utiliser un cookie HttpOnly, Secure et SameSite, avec une protection CSRF. Le limiteur de requêtes devrait aussi utiliser un stockage partagé si plusieurs instances du backend sont déployées.

3. Plan de tests

Niveau	Outil	Nombre
Backend unitaire et HTTP	Bun Test	99
Frontend unitaire	Vitest	18
Intégration PostgreSQL	Bun Test	3
Intégration MongoDB	Bun Test	2
Parcours E2E Chromium	Playwright	2

Les tests unitaires couvrent les règles métier, la validation et les droits. Les tests d'intégration utilisent PostgreSQL et MongoDB réels. Playwright vérifie les parcours depuis l'interface. GitHub Actions exécute le format, le lint, les types, les tests, les builds et la recette Docker.

Les 99 tests backend standards n'incluent pas les 5 intégrations. Le total backend est donc de 104 tests avec les intégrations.

Une exécution est réussie lorsqu'aucune assertion n'échoue, que Svelte Check ne signale ni erreur ni avertissement et que les builds se terminent. Les tests de charge, l'accessibilité complète et Firefox/WebKit restent à ajouter.

4. Jeu d'essai de la fonctionnalité la plus représentative

4.1. Fonctionnalité testée

Le parcours choisi est une réservation publique avec invitation, acceptation et check-in par QR code. Il couvre l'authentification, les droits, la capacité, PostgreSQL et l'interface Svelte.

4.2. Description des scénarios

Étape	Scénario	Résultat attendu
Étape 1	Créer une réservation sur un créneau libre	HTTP 201
Étape 2	Envoyer deux créations identiques en même temps	Une réussite et un HTTP 409
Étape 3	Créer un créneau qui chevauche une réservation	HTTP 409 BOOKING_OVERLAP
Étape 4	Inviter un utilisateur lorsqu'une place est libre	Participant PENDING
Étape 5	Rejoindre une réservation privée sans invitation	HTTP 403
Étape 6	Accepter l'invitation avec le compte invité	Participant ACCEPTED
Étape 7	Générer le QR avec le propriétaire	HTTP 200
Étape 8	Faire le check-in avant le créneau	HTTP 409
Étape 9	Faire le check-in pendant le créneau	HTTP 200 et checkedInAt renseigné
Étape 10	Annuler avec un autre utilisateur standard	HTTP 403
Étape 11	Annuler avec le propriétaire	HTTP 200 et tentative d'audit

4.3. Résultats des tests

Étape	Résultat obtenu
Étape 1	Conforme, réservation et propriétaire enregistrés.
Étape 2	Conforme, PostgreSQL bloque la seconde création.
Étape 3	Conforme, chevauchement refusé avec HTTP 409.
Étape 4	Conforme, invitation créée avec le statut PENDING.
Étape 5	Conforme, accès refusé avec HTTP 403.
Étape 6	Conforme, statut ACCEPTED enregistré.
Étape 7	Conforme, jeton créé et hash stocké.
Étape 8	Conforme, check-in anticipé refusé avec HTTP 409.
Étape 9	Conforme, présence enregistrée dans PostgreSQL.
Étape 10	Conforme, annulation refusée avec HTTP 403.
Étape 11	Conforme, réservation supprimée et audit tenté.

Les scénarios sont couverts par les tests de service, les tests de routes, les intégrations et Playwright. Le parcours E2E accepte l'invitation dans l'interface, ouvre l'URL du QR et vérifie le titre « Présence confirmée ».

4.4. Conclusion

Tous les scénarios sont conformes en local et dans GitHub Actions. Aucun correctif bloquant n'est nécessaire. Les réserves concernent le stockage du JWT, le partage possible du QR, l'absence de tests de charge et l'audit MongoDB en mode best effort.